

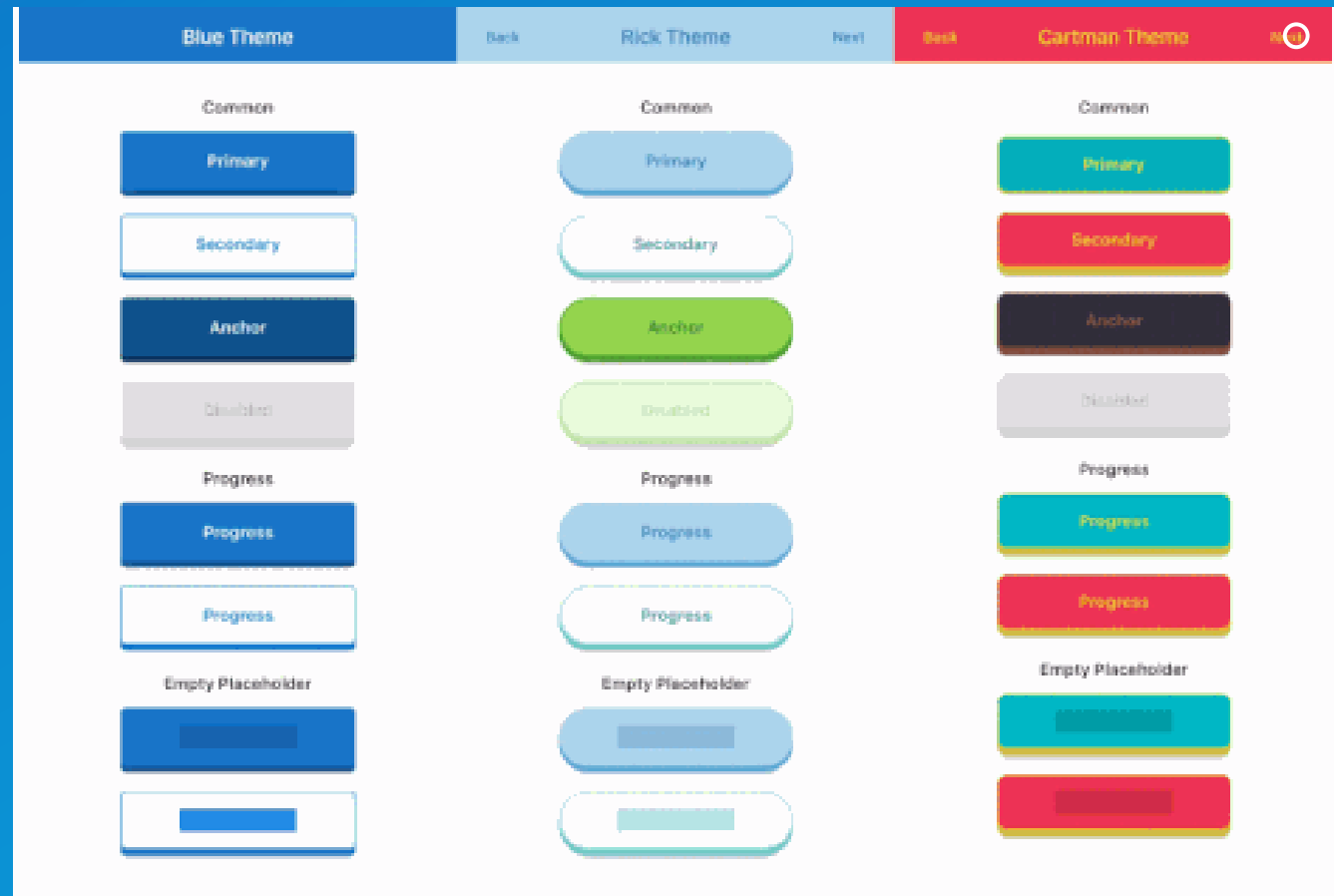


Asztali és mobil alkalmazások fejlesztése és tesztelése tantárgy – Mobil alkalmazások rész

13. A osztály

Tanárok: Dr. Koncz Adrienn és Simon Martin

Gombtípusú komponensek React Native-ban



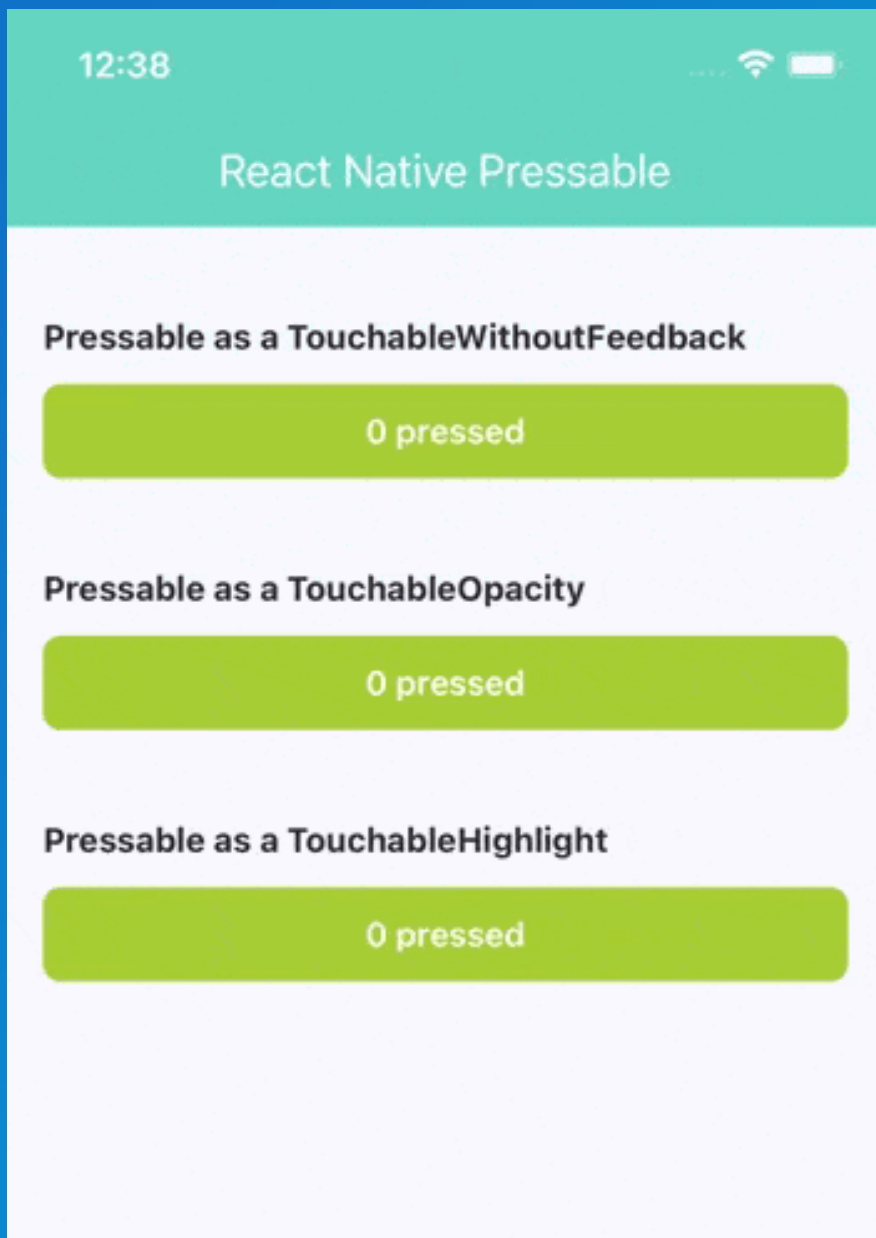
+

•

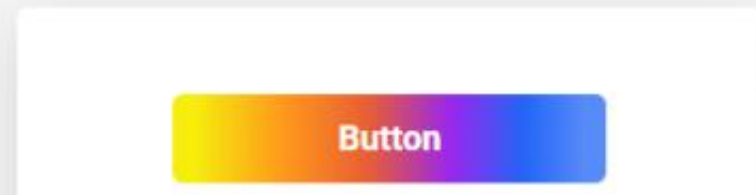
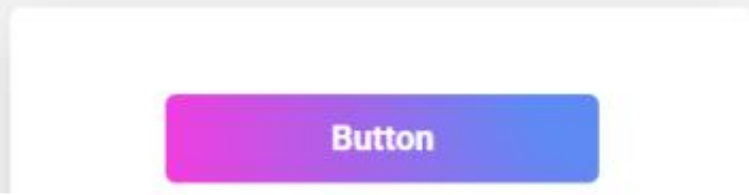
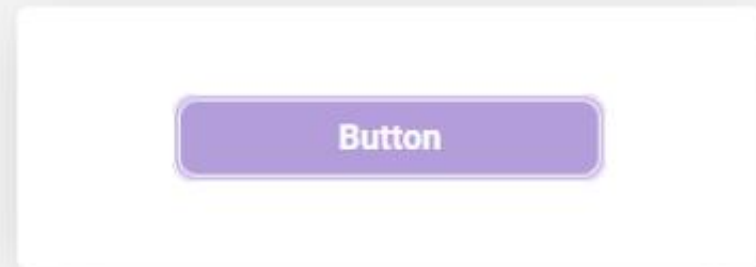
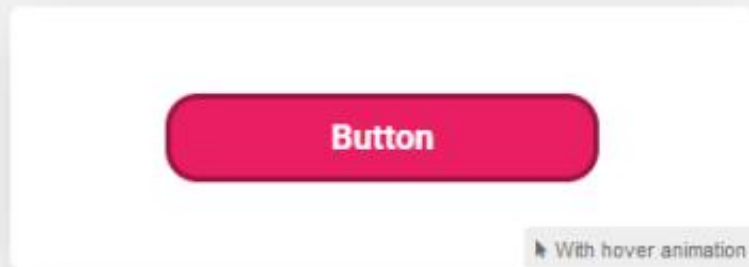
○

TouchableOpacity

- A TouchableOpacity egy olyan React Native komponens, amit akkor használunk, ha szeretnénk, hogy egy elem érintésre reagáljon – tehát például, ha egy gombot, képet vagy ikont kattinthatóvá akarunk tenni.
- A neve is erre utal:
 - Touchable = érinthető
 - Opacity = áttetszőség



- Amikor a felhasználó rákattint vagy megérinti az elemet, a `TouchableOpacity` automatikusan kissé elhalványítja azt, majd visszaállítja az eredeti állapotába.
- Ez egy beépített animáció, ami segít abban, hogy a felhasználó lássa: a rendszer észrevette a kattintást.
- Ez nagyon fontos a mobilos felhasználói élmény szempontjából, hiszen ha semmilyen visszajelzést nem kapunk, bizonytalan lesz, hogy sikerült-e a kattintás.
- <https://reactnative.dev/docs/touchableopacity>



TouchableOpacity stílusa és megjelenése

- A TouchableOpacity nem tartalmaz alapértelmezett stílust – vagyis teljesen üres „vászonként” működik.
- Ezt mi tudjuk formázni stílusokkal (`style={{}}`), így bármilyen színt, méretet, szegélyt, árnyékot vagy animációt adhatunk hozzá.
- Ettől lesz ez a komponens nagyon népszerű, mert szinte bármilyen dizájnú gombot vagy kattintható elemet meg lehet vele csinálni – sokkal rugalmasabb, mint a sima Button.

Hogyan működik a TouchableOpacity a háttérben?

- A TouchableOpacity a gyerekkomponenst (tehát amit beleteszel, például egy Text vagy Image elemet) egy Animated.View-be csomagolja, és az áttetszőséget ennek segítségével változtatja.
- Ez a megoldás azért jó, mert a React Native így egyszerűen tudja animálni az elemet, viszont érdemes tudni, hogy ez néha befolyásolhatja az elrendezést – például ha bonyolultabb komponenseket tartalmaz.



Mikor érdemes használni?

- Ha gombot akarsz készíteni
- ha egy képet vagy ikont kattinthatóvá tennél
- vagy ha egy elemhez érintési visszajelzést szeretnél adni.

A TouchableOpacity előnyei:

- egyszerűen használható,
- jól testreszabható,
- vizuálisan is barátságos visszajelzést ad.





TouchableWithoutFeedback

- A TouchableWithoutFeedback egy React Native komponens, amely lehetővé teszi, hogy egy elem, például egy View, reagáljon az érintésre, de nem ad automatikus vizuális visszajelzést.
 - Ez azt jelenti, hogy ha rákattintunk, nem halványul el, nem vált színt, és nem jelenik meg semmilyen animáció.
 - Akkor érdemes használni, ha nem szeretnénk automatikus effekteket, hanem mi magunk akarjuk meghatározni, hogyan reagáljon az elem az érintésre.
 - Például, ha egy képernyő üres részére kattintva el szeretnénk tüntetni a billentyűzetet, ez a komponens ideális választás lehet.
-

Mikor és miért használjuk a TouchableWithoutFeedback komponenst?

- A TouchableWithoutFeedback olyan helyzetekben hasznos, amikor egy elemre ugyan reagálni szeretnénk, de **nem akarunk látványos animációt vagy stílusváltozást**.
- Ilyen például az, amikor az alkalmazásban csak azt szeretnénk érzékelni, hogy a felhasználó rányomott valamire, de nem akarjuk, hogy ezt a felhasználó vizuálisan is lássa.
- A React Native hivatalos dokumentációja viszont azt javasolja, hogy ezt a komponenst csak indokolt esetben használjuk:
 - *„Do not use unless you have a very good reason”*– vagyis csak akkor, ha valóban szükség van arra, hogy ne legyen automatikus visszajelzés.

További fontos tudnivalók a komponensről

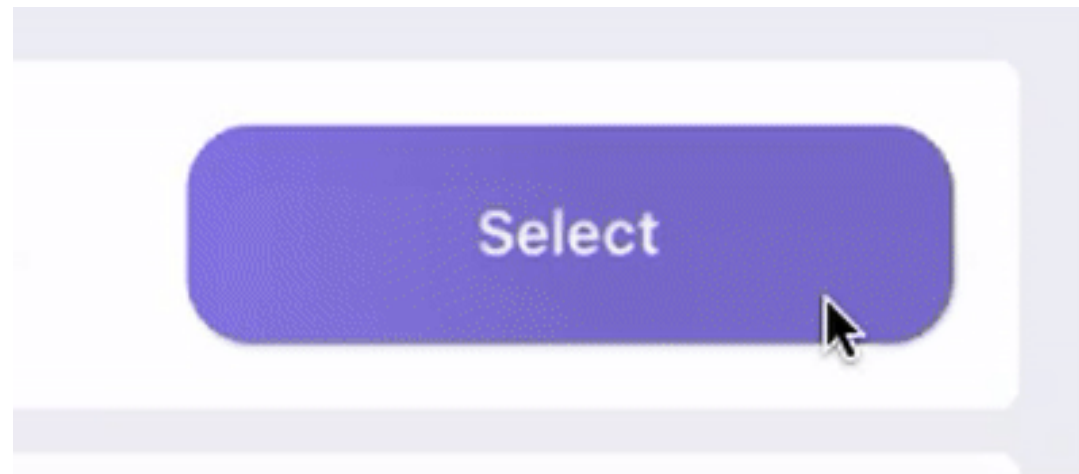
- A komponens csak egyetlen gyermeket tud tartalmazni.
- Ha több elemet szeretnél belerakni, azokat egy View komponensbe kell becsomagolni.
- A működése úgy néz ki, hogy a komponens lemásolja (klónozza) a benne lévő elemet, és hozzáadja az érintéskezeléshez szükséges beállításokat.
- Ha a gyerek nem közvetlenül natív React Native elem (pl. View, Text, Image), hanem egy saját komponens, akkor annak tovább kell adnia ezeket a beállításokat a belső elemnek.
- Ha ez nem történik meg, az érintés nem fog működni.



TouchableHighlight

- A TouchableHighlight egy olyan burkoló (wrapper) komponens, amely lehetővé teszi, hogy a nézetek (views) megfelelően reagáljanak az érintésekre.
- Amikor a felhasználó lenyomja az elemet, a belső nézet áttetszőbbé válik, így láthatóvá válik az alatta lévő „aláfestő” szín (underlay).
- Ez a hatás sötétebbé vagy árnyaltabbá teszi az elemet a nyomás pillanatában.
- Az „underlay” egy új View komponens hozzáadásával jön létre, amely körbefogja a gyermeket.
- Ez azonban befolyásolhatja az elrendezést, és vizuális hibákat is okozhat, ha nem megfelelően használjuk, például ha a belső View háttérszíne nincs kifejezetten átlátszatlan (nem átlátszó) színre állítva.

- A TouchableHighlight csak egyetlen gyermeket tartalmazhat (nem lehet nulla vagy több).
- Ha több elemet szeretnél elhelyezni benne, azokat egy View komponensbe kell becsomagolni.
- <https://reactnative.dev/docs/touchablehighlight>



TouchableHighlight activeOpacity tulajdonsága

- Az activeOpacity határozza meg, milyen mértékben legyen áttetsző a burkolt elem, amikor éppen lenyomják.
- Az értéke 0 és 1 közötti szám lehet (ahol a 1 = teljesen átlátszatlan, a 0 = teljesen átlátszó).
- Alapértelmezett értéke: 0.85.
- Fontos: az activeOpacity működéséhez meg kell adni egy underlayColor értéket is, vagyis azt a színt, ami lenyomáskor az elem alól látszani fog.





Pressable - a modern gomb komponens

- A Pressable egy beépített (Core) React Native komponens, amely képes különböző érintési (megnyomási) eseményeket érzékelni a benne lévő elemeknél.
- Lényegében egy „okos” burkoló (wrapper), ami részletesen kezeli az érintés folyamatát, nemcsak azt, hogy „megnyomták” az elemet.

Hogyan működik a Pressable?

Ha egy elemet Pressable komponensbe ágyazunk, akkor az több eseményt is képes felismerni:

- **onPressIn** – akkor hívódik meg, amikor a felhasználó megérinti / lenyomja az elemet.
- **onPressOut** – akkor fut le, amikor a felhasználó felengedi az ujját.

Ezen felül a lenyomás hosszától függően kétféle dolog történhet:

Ha a felhasználó **gyorsan megérinti és elengedi** az elemet:

- először lefut az **onPressIn**, majd az **onPressOut**, végül az **onPress** esemény.

Ha a felhasználó **legalább fél másodpercig (500 ms) lenyomva tartja** az elemet:

- az **onLongPress** esemény is lefut, majd az **onPressOut** az ujj elengedésekor.

Így a Pressable képes különbséget tenni a rövid érintés és a hosszú lenyomás között.

Miért hasznos a Pressable működése?

- Az emberi ujjak nem túl precízek – gyakran előfordul, hogy a felhasználó kicsit mellényúl, vagy túl kicsi az érinthető terület.
- A Pressable ezért két extra beállítást kínál, amelyekkel az érintési terület finomítható:
 - **HitRect (érzékelési téglalap):** Ezzel megadhatod, **mekkora körzetben érzékelje az érintést az elem körül**. Például ha a gomb kicsi, a **hitSlop** tulajdonsággal **megnővelheted az aktív zónát**, így az érintés akkor is működik, ha a felhasználó egy kicsit mellényúl.
 - **PressRect (nyomástartási téglalap):** Ez azt határozza meg, hogy **mekkora mozgásteret engedünk az ujjnak lenyomás közben**. Ha a felhasználó például lenyom egy gombot, majd kicsit elmozdítja az ujját, az interakció **még mindig érvényesnek számít**. Ezt a tulajdonságot a **pressRetentionOffset** értékkel lehet beállítani.

Fontos tudnivalók

- Az érintési terület soha nem terjedhet túl a szülőnézet határain.
- Ha két elem átfedi egymást, mindig az kapja meg az érintést, amelyik előrébb van (magasabb Z-index).
- <https://reactnative.dev/docs/pressable>



Felső gomb

Fontosabb Pressable tulajdonságok

delayLongPressMegadja: hány ezredmásodperc (milliszekundum) teljen el az **onPressIn** esemény után, mielőtt az **onLongPress** lefutna.

- Ez tehát azt szabályozza, milyen hosszú ideig kell lenyomva tartani az elemet, hogy hosszú nyomásnak számítson.
- Típus: number
- Alapértelmezett érték: 500 (fél másodperc)
- Példa: ha `delayLongPress={1000}`, akkor 1 másodpercig kell nyomva tartani.

disabled: Azt határozza meg, hogy az érintés le legyen tiltva az elemre. Ha `true`, akkor az elem nem reagál semmilyen érintésre.

- Típus: boolean
- Alapértelmezett: `false`

hitSlop

- Ezzel a beállítással **megnövelhetjük az érinthető területet** az elem körül. Hasznos például kis méretű gomboknál, mert így az elem körüli kis távolságban történő érintés is érvényes lesz.
- Típus: Rect vagy number
- Példa: hitSlop={10} azt jelenti, hogy az érintési zóna 10 ponttal megnő minden irányban.

onLongPress

- Akkor fut le, ha a felhasználó **tovább nyomva tartja az elemet**, mint a delayLongPress-ben megadott idő (alapértelmezés: 500 ms).
- Típus: függvény, pl. ({ nativeEvent:PressEvent }) => void
- Példa: hosszan tartó nyomásra törölhetünk egy elemet vagy megnyithatunk egy extra menüt.

onPress

- Akkor hívódik meg, **miután az onPressOut lefutott**, vagyis amikor a felhasználó lenyomta és elengedte az elemet.
- **Típus:** ({ nativeEvent:PressEvent }) => void
- Ez a „normál kattintás” esemény.

onPressIn

- Ez az esemény **azonnal lefut**, amikor az érintés elkezdődik, tehát **még az előtt**, hogy az ujjat elengednénk.
- **Típus:** ({ nativeEvent:PressEvent }) => void
- Gyakran használják animáció indítására, pl. a gomb sötétítésére érintéskor.

onPressMove

- Akkor fut le, amikor az ujj **mozgásban van**, miközben az elem le van nyomva.
- **Típus:** ({ nativeEvent:PressEvent }) => void
- Példa: egy csúszka (slider) esetében ezzel lehet követni, ahogy a felhasználó húzza az ujját.

onPressOut

- Akkor fut le, amikor a felhasználó **elengedi az ujját** — tehát az érintés befejeződik.
- **Típus:** ({ nativeEvent }) => void
- Ez mindig lefut, még akkor is, ha hosszú nyomás után engedjük fel az elemet.

pressRetentionOffset

- Ez határozza meg, **mekkora távolságban** engedhetjük elmozdulni az ujjat, mielőtt az érintés már nem számítana „érvényes nyomásnak”.
- **Típus:** Rect vagy number
- **Alapértelmezett:** { top: 20, bottom: 30, left: 20, right: 20 }
- Magyarul: ha a felhasználó lenyomja az elemet, és az ujját kicsit elcsúsztatja, akkor az érintés még érvényes marad ebben a távolságban.

style

- A Pressable stílusa lehet:
- **statikus** (egyszerű StyleSheet objektum),
- vagy **függvény**, amely megkapja a pressed állapotot, és dinamikusan visszaad egy stílust.
- Ez a módszer ideális, ha a gomb lenyomáskor pl. elsötétedik vagy megváltoztatja a színét.

```
<Pressable
  style={({ pressed }) => [
    styles.button,
    { backgroundColor: pressed ? '#1976D2' : '#2196F3' },
  ]}
  onPress={() => console.log('Megnyomva!')}>
  <Text style={styles.text}>Nyomj meg!</Text>
</Pressable>
```

Ebben a példában a gomb színe megváltozik, amíg le van nyomva, majd visszaáll az eredetire.

Mi az a Rect?

- A Rect olyan numerikus pixelértékeket fogad el, amelyek megadják, hogy egy téglalap-területet mennyivel kell kiterjeszteni.
- Ezek az értékek az eredeti terület méretéhez adódnak hozzá.
- **Kulcsok és értékek:**
 - bottom — típus: szám (number), vagy null, vagy undefined. Kötelező? Nem.
 - left — típus: szám (number), vagy null, vagy undefined. Kötelező? Nem.
 - right — típus: szám (number), vagy null, vagy undefined. Kötelező? Nem.
 - top — típus: szám (number), vagy null, vagy undefined. Kötelező? Nem.

A Rect-et használhatjuk például a következő komponensek:

- Image reactnative.dev
- Pressable reactnative.dev
- Text reactnative.dev
- TouchableWithoutFeedback

```
{  
  bottom: 20,  
  left: null,  
  right: undefined,  
  top: 50  
}
```